BCPD Python Developer

Practice-exam questions and answers

133 questions

Correct options are marked in **green**.

These are the practice-exam questions. Four keys disagreed with the interpreter when the code was run and are corrected here, each one saying what changed. There are no GUI Programming questions in this set, so it does not cover that part of the exam.

Practice-exam set (133)

Domain 1 — Introduction and Setup (37)

1.

What will the value of the i variable be when the following loop finishes its execution?

```
for i in range(10):
    pass
```

A. 10
B. the variable becomes unavailable
C. 11
D. 9

Answer: D

*A `for` loop leaves its control variable holding the **last value it took**, and `range(10)` stops at 9. The variable is still in scope after the loop - Python has no block scope - which is why B is wrong.*

2.

The following expression 1+-2 is:

A. equal to 1
B. invalid
C. equal to 2
D. equal to -1

Answer: D

`1+−2` parses as `1 + (−2)`. The second − is a unary minus attached to the 2, not a second operator, so this is perfectly valid and gives -1.

3.

A compiler is a program designed to

A. rearrange the source code to make it clearer
B. check the source code in order to see if it's correct
C. execute the source code
D. translate the source code into machine code

Answer: B, D

A compiler checks the source for correctness and translates it to machine code. It does not execute the result - that is the job of the program it produced - and it does not reformat your source.

4.

What is the output of the following piece of code?

```
a='ant'
b='bat'
c='camel'
print(a,b,c,sep='')
```

A. ant' bat' camel
B. ant'bat'camel
C. antbatcamel
D. SyntaxError

Answer: C

*`sep` controls what `print` puts *between* its arguments, and here it is set to an empty string, so the three words run together as `antbatcamel`. The default `sep` is a single space.*

5.

What is the expected output of the following snippet?

```
i=5
while i>0:
    i=i//2
    if i%2==0:
        break
    else:
        i+=1
print(i)
```

A. the code is erroneous
B. 3
C. 7
D. 15

Answer: A

This question is broken. *The code runs fine and prints **2**, but none of the four options offers 2 - and option A claims the code is erroneous, which it is not. `i=5` becomes `5//2 = 2`, then `2 % 2 == 0` is true so `break` fires immediately.*

*Traced by running it: `i = 5 // 2` is 2, and `2 % 2 == 0` is true, so `break` fires on the first pass and `print(i)` shows **2**.*

6.

How many lines does the following snippet output?

```
for i in range(1,3):
    print('**',end='')
else:
    print('**')
```

A. three
B. one
C. two
D. four

Answer: B

This question's key was wrong and is corrected here. *It asks how many *lines* are printed. `print('**', end='')` runs twice and emits no newline, then the loop's `else` runs `print('**')` which does. The whole output is `******` followed by one newline - **one line**, not two.*

`print('**', end='')` suppresses the newline, so the two loop passes emit `****` on one line. A `while`/`for` `else` block runs when the loop finishes without `break`, adding `**` and a newline. One line of six stars.

7.

Which of the following literals reflect the value given as 34.23?

A. 3.42E+01
B. 3.42E+01
C. 3.42E-03
D. 3.42E+05

Answer: A, B

This question is broken. *Options A and B are byte-identical (`3.42E+01`) and both are keyed correct, so one of them is a typo for something else. On the content: 34.23 is 3.423E+01, so `3.42E+01` is the intended form.*

34.23 in scientific notation is 3.423 x 10^1, written `3.423E+01`. The exponent is the number of places the point moves.

8.

Select the valid fun() invocations:

```
def fun(a,b=0):
    return a*b
```

A. fun(b=1)
B. fun(a=0)
C. fun(b=1,0)
D. fun(1)

Answer: B, D

b has a default so it may be omitted, but a has none and must be supplied. `fun(b=1)` leaves a unset. `fun(b=1, 0)` is a syntax error - a positional argument can never follow a keyword one.

9.

What is the expected output of the following code?

```
def f(n):
    if n==1:
        return '1'
    return str(n)+f(n-1)
print(f(2))
```

A. 21
B. 12
C. 3
D. 1

Answer: A

`f(2)` returns `str(2) + f(1)`, and `f(1)` returns `'1'`. String concatenation, not addition, so the result is the text `21`.

10.

What is the expected behaviour of the following snippet?

```
def x():
    return 2
x=1+x()
print(x)
```

A. cause a runtime exception on line 02
B. cause a runtime exception on line 01
C. cause a runtime exception on line 03
D. print 3

Answer: D

The `def` runs first and binds x to the function; then `x = 1 + x()` calls it while the name still refers to the function, gets 2, adds 1, and rebinds x to 3. Rebinding a name is always legal in Python.

11.

What is the expected behaviour of the following code?

```
def f(n):
    for i in range(1,n+1):
        yield i
print(f(2))
```

A. print 4321
B. print <generator object f at (some hex digits)>
C. cause a runtime exception
D. print 1234

Answer: B

*A function containing `yield` is a **generator function**. Calling it does not run the body - it returns a generator object, which is what gets printed. You would need `list(f(2))` or a `for` loop to see the values.*

12.

If you need a function that does nothing, what would you use instead of XXX?

```
def idler(): XXX
```

A. pass
B. return
C. exit
D. None

Answer: A, B

Both `pass` and `return` are valid single-statement bodies. `pass` is the explicit do-nothing statement; a bare `return` exits immediately, also returning `None`. `exit` would end the program and `None` is a value, not a statement.

13.

Which of the following words can be used as a variable name?

A. for
B. TRUE
C. TRUE
D. For

Answer: C, D

***Options B and C are identical** (TRUE), so one is a typo for something else. The answer still holds: `for` is a reserved keyword and cannot be a name, while TRUE and `For` are ordinary identifiers - Python's boolean is spelled `True`, so TRUE is just a name, and names are case-sensitive.*

`for` is a reserved keyword, so it cannot be a name. `For` is a different identifier - names are case-sensitive - and TRUE is an ordinary name too, since Python's boolean is spelled `True`.

14.

Python strings can be 'glued' together using the operator:

A. &
B. *
C. ",
D. +

Answer: D

*+ concatenates strings. * repeats one ('ab' * 3), and & is bitwise AND, which strings do not support.*

15.

A keyword

A. can be used as an identifier
B. is defined by Python's lexis
C. is also known as a reserved word
D. cannot be used in the user's code

Answer: B, C

A keyword is part of the language's own lexis - a reserved word - so it cannot be used as an identifier. It very much does appear in your code; that is the point of `if`, `for` and `def`.

16.

How many stars (*) does the snippet print?

```
S='***********'
print(s)
```

A. the code is erroneous
B. five
C. four
D. None (11 stars)

Answer: A

Python is case-sensitive, so `S` and `s` are different names. `S` was assigned, `s` never was, and `print(s)` raises `NameError`.

17.

Assuming that V holds integer 2, which operator should be used instead of OPER to make V OPER 1 equal to 1?

A. «<
B. »>
C. »
D. «

Answer: C

» shifts bits right, and shifting binary 10 (decimal 2) right by one gives binary 1. There is no »> operator in Python.

18.

How many stars (*) does the following snippet print?

```
i=3
while i>0:
    i-=1
    print('*')
else:
    print('*')
```

A. the code is erroneous
B. five
C. three
D. four

Answer: D

The loop runs three times, printing a star each pass, and the `else` clause runs once when the loop ends without `break` - four stars in total. Verified by running it.

19.

UNICODE is:

A. the name of an operating system
B. a standard for encoding and handling texts

C. the name of a programming language
D. the name of a text processor

Answer: B

Unicode is a standard that assigns a number, called a code point, to every character in every writing system, plus encodings such as UTF-8 that store those numbers as bytes.

20.

Which of the equations are True?

A. chr(ord(x)) == x
B. ord(ord(x)) == x
C. chr(chr(x)) == x
D. ord(chr(x)) == x

Answer: A, D

`ord()` turns a character into its code point and `chr()` turns a code point back into a character, so each undoes the other. The other two combinations pass the wrong type in.

21.

A two-parameter lambda function raising its first parameter to the power of the second parameter should be declared as:

A. lambda (x,y) = x**y
B. lambda (x,y): x**y
C. def lambda (x,y): return x**y
D. lambda x, y: x**y

Answer: D

A lambda is written `lambda params: expression` - no parentheses around the parameters, no `return`, no `def`. The expression's value is what it returns.

22.

What is the expected output of the following code?

```
def f(n):
    if n==1:
        return 1
    return n+f(n-1)
print(f(2))
```

A. 21
B. 12
C. 3
D. none

Answer: C

*`f(2)` returns `2 + f(1)`, and `f(1)` hits the base case and returns 1. These are numbers, so it adds to **3** - compare with the near-identical question that concatenates strings and gives `21`.*

23.

Which function or operator should you use to obtain the answer True or False to the question: 'Do two variables refer to the same object?'

A. The = operator
B. The isinstanceOf function
C. The id() function
D. The is operator

Answer: D

`is` compares identity - whether two names point at the same object. `==` compares value, and = assigns. `id()` gives you the identity number but you would have to compare the two results yourself.

24.

Select the true statements related to PEP 8 programming recommendations.

A. You should use the not ... is operator rather than the is not operator
B. You should make object type comparisons using the isinstance() method
C. You should write code in a way that favours the CPython implementation over PyPy, Cython, and Jython
D. You should not write string literals that rely on significant trailing whitespace

Answer: B, D

PEP 8 says to use `isinstance()` for type comparisons and warns against trailing whitespace in literals, which is invisible and easily lost. It prefers `is not` over `not ... is`, and explicitly tells you not to favour one implementation.

25.

Which of the following statements are true?

A. ASCII stands for Information Interchange
B. a code point is a number assigned to a given character
C. ASCII is synonymous with UTF-8
D. \e is an escape sequence used to mark the end of lines

Answer: A, B

*A code point is the number assigned to a character. ASCII is *American Standard Code for Information Interchange* - the option here drops the first half. UTF-8 is a Unicode encoding, not a synonym for ASCII, and `\n` marks a line end, not `\e`.*

26.

What is the expected output of the following code?

```
myTu=('a','b','c')
m=tuple(map(lambda x: chr(ord(x)+1), myTu))
print(m[-2])
```

A. a
B. c
C. an exception is raised
D. b

Answer: B

This question's key was wrong and is corrected here. *The bank keys b; running it gives c.*

*The bank keys **b**. Running it: `('a','b','c')` maps through `chr(ord(x)+1)` to `('b','c','d')`, so `m[-2]` - the second from the end - is **c**.*

27.

Assuming that the following code has been executed successfully, which of the expressions evaluate to True?

```
def f(x,y):
    nom,denom=x,y
    def g():
        return nom/denom
    return g
a=f(1,2)
b=f(3,4)
```

A. b()==4
B. a!=b
C. a is not None
D. a()==4

Answer: B, C

`f` returns a new function each call, so a and b are different objects and neither is `None`. Each closure remembers its own `nom` and `denom`, so `a()` is 0.5 and `b()` is 0.75.

28.

What is the expected behaviour of this code?

```
def foo(x,y):
    return y(x)+(x+1)
print(foo(1, lambda x: x*x))
```

A. 3
B. 5
C. 4
D. an exception is raised

Answer: A

*`y(x)` is `1*1 = 1`, then `+ (x+1)` adds 2, giving **3**. Note the second term is not passed through the lambda - compare the near-identical question where it is.*

29.

Which of the following lambda definitions are correct?

A. lambda x,y: (x,y)
B. lambda x,y: return x/y - x%y
C. lambda x,y: x/y - x%y
D. lambda x,y = x/y - x%y

Answer: A, C

A lambda's body is a single expression, so there is no `return` and no =. Returning a tuple is fine because `(x, y)` is an expression.

30.

What is the expected behaviour of this code?

```
x = 3 % 1
y = 1 if x > 0 else 0
print(y)
```

A. the code is erroneous and it will not execute
B. it outputs 1
C. it outputs -1
D. it outputs 0

Answer: D

`3 % 1` is 0 - one divides into three exactly, leaving no remainder. So `x > 0` is false and the conditional expression yields 0.

31.

What is the expected behaviour of this code?

```
x = 8 ** (1/3)
y = 2. if x < 2.3 else 3.
print(y)
```

A. the code is erroneous and it will not execute
B. it outputs 2.0
C. it outputs 2.5
D. it outputs 3.0

Answer: B

`8 ** (1/3)` is the cube root of 8, which is 2.0. `2.0 < 2.3` is true, so the conditional expression yields `2.` - and `2.` is a float literal, so it prints as `2.0`.

32.

Which of the following statements are true?

A. an escape sequence can be recognised by the / sign put in front of it
B. ASCII stands for Internal Information
C. ASCII is a subset of UNICODE
D. a code point is a number assigned to a given character

Answer: C, D

ASCII's 128 characters are the first 128 code points of Unicode, so it is a subset, and a code point is the number assigned to a character. An escape sequence starts with a backslash, not a slash.

33.

What is the expected behaviour of this code?

```
string='123'
dummy=0
for character in reversed(string):
    dummy += int(character)
print(dummy)
```

A. it outputs 321
B. it outputs 123
C. it outputs 6
D. it raises an exception

Answer: C

`reversed()` walks the characters backwards, but addition does not care about order - 3 + 2 + 1 is the same 6. The reversal is a red herring.

34.

Which of the following expressions evaluate to True?

A. ord('2') - ord('2') == ord('0')
B. len('6E6E6E6E') > 0
C. chr(ord('a')+1) == 'B'
D. len('') == 1

Answer: A, B

This question is broken. *It asks for two, but only B is true. Option A - `ord('2') - ord('2') == ord('0')` - is 0 == 48, which is* ***false****, and it is keyed correct. `chr(ord('a')+1)` is `'b'`, not `'B'`, and an empty string has length 0.*

`ord('2') - ord('2')` is 0, and `ord('0')` is 48 - so A is false as written and this key is worth checking. B is true: an eight-character string has a length above 0. `chr(ord('a')+1)` is `'b'`, not `'B'`, and an empty string has length 0.

35.

What is the expected behaviour of this code?

```
def foo(x,y):
    return y(x)+y(x+1)
print(foo(1, lambda x: x*x))
```

A. 4
B. 3
C. an exception is raised
D. 5

Answer: D

`y(x)` is `1*1 = 1` and `y(x+1)` is `2*2 = 4`, so the sum is ***5****. Both terms go through the lambda here - compare the near-identical question where the second does not.*

36.

Which of the following expressions evaluate to True?

A. ord('2') - ord('2') == ord('0')
B. chr(ord('A')+1) == 'B'
C. len('') == 1
D. len('0') == ('E'*'E')

Answer: A, B

This question is broken. *Same defective option as the other* `ord` *question:* `ord('2') - ord('2') == ord('0')` *is 0 == 48, which is* ***false****, and it is keyed correct. Only B is true.*

`chr(ord('A')+1)` *is* `'B'` *-* `ord` *gives the code point, +1 moves one letter on,* `chr` *converts back.* `'E'*'E'` *would raise* `TypeError`*, since you cannot multiply two strings.*

37.

Which of the following is not a version of Python?

A. Python 2
B. Python 3
C. Python X
D. Python 4

Answer: C

There is no Python X. The two versions that matter are Python 2, now end-of-life, and Python 3, which is what you should be learning.

Domain 2 — Data Structures (20)

38.

Assuming that the following snippet has been successfully executed, which of the equations are True?

```
a=[1]
b=a
a[0]=0
```

A. len(a)==len(b)
B. b[0]+1==a[0]
C. a[0]==b[0]
D. a[0]+1==b[0]

Answer: A, C

`b = a` *does not copy - both names point at the same list. Changing* `a[0]` *changes what* `b[0]` *shows, so the two are always equal and always the same length.*

39.

Assuming that the following snippet has been successfully executed, which of the equations are False?

```
a=[0]
b=a[:]
a[0]=1
```

A. len(a)==len(b)
B. a[0]-1==b[0]
C. a[0]==b[0]
D. b[0]-1==a[0]

Answer: C, D

`b = a[:]` ***does*** *copy, so the two lists are now independent. Setting* `a[0] = 1` *leaves* `b[0]` *at 0, which makes* `a[0] == b[0]` *false and* `b[0] - 1 == a[0]` *false (that would be -1 == 1).*

40.

Which of the following statements are false?

A. Python strings are actually lists
B. Python strings can be concatenated
C. Python strings can be sliced like lists
D. Python strings are mutable

Answer: A, D

*Strings are their own type, not lists, and they are **immutable** - you cannot assign to `s[0]`. They can be concatenated and sliced, which is what makes them feel list-like.*

41.

Which of the following sentences are true?

A. Lists may not be stored inside tuples
B. Tuples may be stored inside lists
C. Tuples may not be stored inside tuples
D. Lists may be stored inside lists

Answer: B, D

*Lists and tuples can each hold the other, and can nest inside themselves. The only real restriction is the other way round: a *list* cannot be a dictionary key or a set member, because it is mutable and therefore unhashable.*

42.

Assuming that String is six or more letters long, the following slice string[1:-2] is shorter than the original string by:

A. four chars
B. three chars
C. one char
D. two chars

Answer: B

`[1:-2]` drops one character from the front and two from the end - three in total. Negative indices count back from the end, and the stop is exclusive.

43.

What is the expected output of the following snippet?

```
lst=[1,2,3,4]
lst=lst[-3:-2]
lst=lst[-1]
print(lst)
```

A. 1
B. 4
C. 2
D. 3

Answer: C

Run it: `lst[-3:-2]` takes one element, the third from the end, giving `[2]`. Then `lst[-1]` on that one-element list is the element itself, 2.

44.

How many elements will the list2 list contain after execution of the following snippet?

```
list1=[False for i in range(1,10)]
list2=list1[-1:1:-1]
```

A. zero
B. five

C. seven
D. three

Answer: C

*`list1` has 9 elements, indices 0 to 8. `[-1:1:-1]` starts at index 8 and steps backwards, stopping *before* index 1, so it takes 8, 7, 6, 5, 4, 3, 2 - **seven**.*

45.

What would you use instead of XXX if you want to check whether a certain key exists in a dictionary called dict?

A. 'key' in dict
B. dict['key'] != None
C. dict.exists('key')
D. 'key' in dict.keys()

Answer: A, D

*`in` on a dictionary tests its **keys**, so `'key' in dict` is the idiomatic form; `'key' in dict.keys()` does the same thing more slowly. There is no `.exists()`, and comparing to None fails when the stored value genuinely is None.*

46.

You need data which can act as a simple telephone directory. You can obtain it with the following clauses

A. dir={'Mom':5551234567,'Dad':5557654321}
B. dir={'Mom':'5551234567','Dad':'5557654321'}
C. dir={Mom:5551234567,Dad:5557654321}
D. dir={Mom:'5551234567',Dad:'5557654321'}

Answer: A, B

*Dictionary **keys** must be written as values - here quoted strings. Without the quotes, Mom is a name Python tries to look up, and raises `NameError`. Both keyed options quote the keys; whether the numbers are strings or ints is free choice.*

47.

What is the expected output of the following code?

```
str='abcdef'
def fun(s):
    del s[2]
    return s
print(fun(str))
```

A. abcef
B. The program will cause a runtime exception/error
C. acdef
D. abdef

Answer: B

Strings are immutable, so `del s[2]` raises `TypeError: 'str' object doesn't support item deletion`. To remove a character, build a new string: `s[:2] + s[3:]`.

48.

Which of the listed actions can be applied to the following tuple?

```
tup=()
```

A. tup[0]
B. tup.append(0)
C. tup[0]
D. del tup

Answer: C, D

***This question is broken.** Options A and C are identical (`tup[0]`), and one of them is keyed correct. On an *empty* tuple `tup[0]` raises `IndexError`, so it cannot be applied at all; `del tup` genuinely can.*

A tuple is immutable, so there is no append and no item assignment. `del tup` deletes the whole name, which is always allowed.

49.

Executing the snippet dct={'pi':3.14}; dct['pi']=3.1415 will cause the dct:

A. to hold two keys named 'pi' linked to 3.14 and 3.1415 respectively
B. to hold two key named 'pi' linked to 3.14 and 3.1415
C. to hold one key named 'pi' linked to 3.1415
D. to hold two keys named 'pi' linked to 3.1415

Answer: C

A dictionary key is unique. Assigning to an existing key replaces its value rather than adding a second entry, so the dictionary still has one key.

50.

How many elements will the list1 list contain after execution of the following snippet?

```
list1 = 'don't think twice, do it!'.split(',')
```

A. two
B. zero
C. one
D. three

Answer: A

`split(',')` cuts at each comma. The text has one comma, so it produces two pieces. Splitting always yields one more piece than the number of separators found.

51.

If you want to transform a string into a list of words, what invocation would you use? (Expected output: The, Catcher, in, the Rye,)

```
S='The Catcher in the Rye'
L=# put proper invocation
```

A. s.split()
B. split(s,' ')
C. s.split(' ')
D. split(s)

Answer: A

*`split()` with no argument splits on any run of whitespace and discards empty pieces, which is what you want for words. It is a *method* on the string, so `s.split()` - there is no bare `split()` function.*

52.

Assuming that lst is a four-element list, is there any difference between these two statements? del lst del lst[:]

A. yes, the first line empties the list, the second deletes the list as a whole
B. yes, the first line deletes the list as a whole, the second just empties the list
C. no, there is no difference
D. yes, the first line deletes the list as a whole, the second removes all elements except the first

Answer: B

`del lst` removes the name entirely, so using it afterwards raises `NameError`. `del lst[:]` deletes every element through a slice, leaving an empty list still bound to the name.

53.

Which of the following expressions evaluate to True?

A. str(1-1) in '123456789'
B. 'dcb' not in 'abcde'[::-1]

C. 'phd' in 'alpha'
D. 'True' not in 'False'

Answer: A, D

This question is broken. *It asks for two true expressions, but only D is true.* `str(1-1)` *is* `'0'`*, and* `'0'` *does not appear in* `'123456789'`*, so A is false - yet A is keyed.* `'abcde'[::-1]` *is* `'edcba'`*, which does contain* `'dcb'`*, and* `'phd'` *is not in* `'alpha'`*.*

`str(1-1)` *is* `'0'`*, which is not in* `'123456789'` *- so A is false as written and this key is worth checking.* `'True' not in 'False'` *is true: it is a substring test, and* `'True'` *does not appear inside* `'False'`*.*

54.

What is the expected behaviour of the following code?

```
the_list='alpha;beta;gamma'.split(':')
the_string='.'.join(the_list)
print(the_string.isalpha())
```

A. it raises an exception
B. it outputs True
C. it outputs False
D. it outputs nothing

Answer: C

`.split(':')` *finds no colon, so it returns the whole string as a single-element list. Joining that gives the original text back, and* `isalpha()` *is False because of the semicolons.*

55.

Assuming that the snippet below has been executed successfully, which of the following expressions evaluate to True?

```
string='python'[::2]
string=string[-1]+string[-2]
```

A. string[0]=='o'
B. string is None
C. len(string)==3
D. string[0]==string[-1]

Answer: A

The stem says choose two but the bank keys only one. A is correct: `'python'[::2]` *is* `'pto'`*, then* `string[-1] + string[-2]` *is* `'ot'`*, so* `string[0]` *is* `'o'`*.*

`'python'[::2]` *takes every second character -* `'pto'`*. Then* `string[-1] + string[-2]` *is* `'o'` *reversed onto* `'t'`*, giving* `'ot'`*, so* `string[0]` *is* `'o'` *and the length is 2.*

56.

Which of the following expressions evaluate to True?

A. 'in' in 'Thames'
B. 'in' in 'in'
C. 'in not' in 'not'
D. 't'.upper() in 'Thames'

Answer: B, D

`in` *is a substring test.* `'Thames'` *does not contain* `'in'`*;* `'in'` *trivially contains itself; and* `'t'.upper()` *is* `'T'`*, which* `'Thames'` *does start with.*

57.

Which of the following invocations are valid?

A. sort('python')
B. 'python'.find('e')
C. 'python'.index('eth')
D. 'python'.sorted()

Answer: B, C

`find()` and `index()` are both real string methods, so both calls are valid syntax - `find` returns -1 when absent and `index` raises. There is no bare `sort()` function for strings and no `.sorted()` method; the built-in is `sorted('python')`.

Domain 3 — Object-Oriented Programming (32)

58.

Is it possible to safely check if a class/object has a certain attribute?

A. yes, by using the hashtart attribute
B. yes, by using the hashtart() method
C. yes, by using the hasattr() function
D. no, it is not possible

Answer: C

`hasattr(obj, 'name')` returns True or False without raising, which is what makes it the safe check. The alternative is `getattr(obj, 'name', default)`.

59.

The first parameter of each method:

A. holds a reference to the currently processed object
B. is always set to None
C. is set to a unique random value
D. is set by the first argument's value

Answer: A

The first parameter receives the instance the method was called on, conventionally named `self`. `obj.method()` is shorthand for `Class.method(obj)`.

60.

The simplest possible class definition in Python can be expressed as:

A. class X:
B. class X: pass
C. class X: return
D. class X: {}

Answer: B

A class body cannot be empty, so it needs at least one statement - `pass` is the null statement that satisfies that. `class X:` alone is a syntax error.

61.

A variable stored separately in every object is called:

A. there are no such variables, all variables are shared among objects
B. a class variable
C. an object variable
D. an instance variable

Answer: D

An instance variable is stored on each object separately, normally assigned as `self.x = ...` inside `__init__`. A class variable lives on the class and is shared by every instance.

62.

The following class hierarchy is given. What is the expected output of the code?

```
class A:
    def a(self):
        print('A',end=' ')
```

```
    def b(self):
        self.a()
...
B().do()
C().do()
```

A. BB
B. CC
C. AA
D. BC

Answer: D

*b() finds b on the base class, which calls `self.a()`. Because `self` is the *subclass* instance, the subclass's own a runs - so each class prints its own letter. That is polymorphism through the method resolution order.*

63.

If any of a class's components has a name that starts with two underscores (__), then:

A. the class component's name will be mangled
B. the class component has to be an instance variable
C. the class component has to be a class variable
D. the class component has to be a method

Answer: A

*Two leading underscores trigger **name mangling** - `__x` inside `class C` becomes `_C__x`. It prevents accidental clashes in subclasses; it is not access control, and the attribute is still reachable under the mangled name.*

64.

A function called issubclass(c1,c2) is able to check if:

A. c1 and c2 are both subclasses of the same superclass
B. c2 is a subclass of c1
C. c1 is a subclass of c2
D. c1 and c2 are not subclasses of the same superclass

Answer: C

`issubclass(c1, c2)` reads in argument order: is the first a subclass of the second. `isinstance` follows the same order with an object first.

65.

A class constructor

A. can return a value
B. cannot be invoked directly from inside the class
C. can be invoked directly from any of the subclasses
D. can be invoked directly from any of the superclasses

Answer: B, C

`__init__` runs automatically when you instantiate; you never call it directly on the class, but a subclass calls the parent's through `super().__init__()`. It must return None.

66.

The following class definition is given. We want the show() method to invoke the get() method, and then output the value the get() method returns. Which invocation should be used instead of XXX?

```
class Class:
    def __init__(self,val):
        self.val=val
    def get(self):
        return self.val
    def show(self):
        XXX
```

A. print(get(self))
B. print(self.get())
C. print(get())
D. print(self.get(val))

Answer: B

Calling another method on the same object needs `self.` in front - `self.get()`. Without it Python looks for a plain function called `get` and raises `NameError`.

67.

What is the expected output of the following snippet?

```
class X: pass
class Y(X): pass
class Z(Y): pass
X=Z()
Z=Z()
print(isinstance(X,Z), isinstance(Z,X))
```

A. True False
B. True True
C. False False
D. False True

Answer: D

This question is broken. *`Z = Z()` rebinds the name Z from a class to an *instance*, so `isinstance(X, Z)` raises `TypeError: isinstance() arg 2 must be a type`. The code never prints anything, and no option says it raises.*

`Z = Z()` rebinds the name Z from the class to an instance of it, so the class is no longer reachable by that name and `isinstance(X, Z)` gets an object where it needs a type.

68.

Which sentence about the @property decorator is false?

A. The @property decorator should be defined after the method that is responsible for setting an encapsulated attribute.
B. The @property decorator designates a method which is responsible for returning an attribute value.
C. The @property decorator marks the method whose name will be used as the name of the instance attribute.
D. The @property decorator should be defined before the methods that are responsible for setting and deleting an encapsulated attribute.

Answer: A

*`@property` goes on the **getter**, and it has to come first because the setter is then written as `@name.setter`, which refers back to the property the getter created. Defining it afterwards would have nothing to attach to.*

69.

Select the true statement about the __name__ attribute.

A. __name__ is a special attribute, inherent for both classes and instances, and it contains information about the class to which a class instance belongs.
B. __name__ is a special attribute, inherent for both classes and instances, and it contains a dictionary of object attributes.
C. __name__ is a special attribute, inherent for classes and it contains information about the class to which a class instance belongs.
D. __name__ is a special attribute, inherent for classes, and it contains the name of a class.

Answer: D

*`__name__` on a class holds the class's own name as a string. The attribute that tells an *instance* which class it belongs to is `__class__`, and the dictionary of attributes is `__dict__`.*

70.

What is the expected behaviour of the following code? (missing colon after Sub_B(Super))

A. it outputs 0
B. it outputs 1
C. it raises an exception
D. it outputs 2

Answer: C

A missing colon after a `class` header is a `SyntaxError`, so the file never runs.

71.

Assuming that the following inheritance set is in force, which of the following classes are declared properly?

```
class A: pass
class B(A): pass
class C(A): pass
class D(B): pass
```

A. class Class_4(D,A): pass
B. class Class_3(A,C): pass
C. class Class_2(B,D): pass
D. class Class_1(C,D): pass

Answer: A, C

A class list is valid when Python can find one consistent method resolution order. Deriving from a class and its own ancestor in the wrong order is what fails - the more specific class has to come first.

72.

What is the expected behaviour of this code?

```
class Upper:
    def method(self):
        return 'upper'
class Lower(Upper):
    def method(self):
        return 'lower'
Object=Upper()
print(isinstance(Object,Lower), end=' ')
print(Object.method())
```

A. True upper
B. True lower
C. False upper
D. False lower

Answer: C

`Object` is an `Upper`, and a parent is never an instance of its child, so `isinstance` is False. The method called is the one on `Upper`, printing `upper`.

73.

What is the expected behaviour of the following code? (o1.foo missing parentheses)

A. it outputs 1
B. it outputs 3
C. it outputs 6
D. it raises an exception

Answer: D

*`o1.foo` without parentheses is the method *object*, not a call. Using it where a number is expected raises `TypeError`.*

74.

What is the expected behaviour of this code?

```
class Class:
    Variable=0
    def __init__(self):
        self.value=0
object_1=Class()
object_1.Variable+=1
object_2=Class()
object_2.value+=1
print(object_2.Variable+object_1.value)
```

A. it outputs 0
B. it raises an exception
C. it outputs 1
D. it outputs 2

Answer: A

*`object_1.Variable += 1` reads the shared class attribute (0), adds one, and stores the result as a **new instance attribute** on `object_1` only. `object_2.Variable` still reads the class's 0, and `object_1.value` is its own 0, so the sum is 0.*

75.

What is true about Object-Oriented Programming in Python?

A. each object of the same class can have a different set of methods
B. a subclass is usually more specialised than its superclass
C. if a real-life object can be described with a set of adjectives, they may reflect a Python object method
D. the same class can be used many times to build a number of objects

Answer: B, D

Every instance of a class has the same methods - that is what the class defines. A subclass specialises its parent, and one class builds as many objects as you like.

76.

What is true about Python class constructors?

A. there can be more than one constructor in a Python class
B. the constructor must return a value other than None
C. the constructor is a method named __init__
D. the constructor must have at least one parameter

Answer: C, D

The constructor is `__init__`, and its first parameter is the instance, so it always has at least one. It must return None, and a second `def __init__` simply replaces the first - Python has no overloading.

77.

Assuming that the following piece of code has been executed successfully, which of the expressions evaluate to True?

```
class A:
    VarA=1
    def __init__(self):
        self.prop_a=1
class B(A):
    VarA=2
    def __init__(self):
        super().__init__()
        self.prop_b=2
obj_a=A()
obj_aa=A()
obj_b=B()
obj_bb=obj_b
```

A. isinstance(obj_b,A)
B. A.VarA==1
C. obj_a is obj_aa
D. B.VarA==1

Answer: A, B

`obj_b` is a B, and B derives from A, so `isinstance(obj_b, A)` is true. `A.VarA` is still 1 - B shadowing it with 2 does not change the parent. `obj_a` and `obj_aa` are separate objects, so `is` is false.

78.

What is the expected behaviour of this code? Which are true?

```
class Class:
    Var=data=1
    def __init__(self, value):
        self.prop=value
Object=Class(2)
```

A. len(Class.__dict__)==1
B. 'data' in Class.__dict__
C. 'var' in Class.__dict__
D. 'data' in Object.__dict__

Answer: B

The stem says choose two but the bank keys only one. `Var = data = 1` binds both names in the class body, so `'data' in Class.__dict__` is true.

*`Var = data = 1` in the class body binds **both** names on the class, so `'data'` is in `Class.__dict__`. `'var'` is not - names are case-sensitive - and `prop` lives on the instance, not the class.*

79.

A property that stores information about a given class's super-classes is named:

A. __upper__
B. __super__
C. __ancestors__
D. __bases__

Answer: D

`__bases__` holds the tuple of a class's direct base classes.

80.

What is the expected behaviour of this code?

```
class Class:
    Var=0
    def _foo(self):
        Class.Var+=1
        return Class.Var
o=Class()
o._Class_foo()
print(o._Class_foo())
```

A. it raises an exception
B. it outputs 3
C. it outputs 1
D. it outputs 6

Answer: A

*Only a **double** underscore triggers name mangling. `_foo` has one, so it is not renamed and `o._Class_foo()` looks for an attribute that does not exist, raising `AttributeError`. Had the method been `__foo`, the mangled name would be `_Class__foo` - note the two underscores in the middle.*

81.

The __bases__ property contains:

A. base class location (addr)
B. base class objects (class)
C. base class names (str)
D. base class ids (int)

Answer: B

*`__bases__` holds the base **class objects** themselves, not their names or addresses. `SubClass.__bases__[0].__name__` is how you would get a name.*

82.

What is the expected behaviour of this code?

```
class Super:
    def make(self): pass
    def doit(self):
        return self.make()
class Sub_A(Super):
    def make(self):
        return 1
class Sub_B(Super):
    pass
a=Sub_A()
b=Sub_B()
print(a.doit()+b.doit())
```

A. It outputs 0
B. It raises an exception
C. It outputs 1
D. It outputs 2

Answer: B

`Sub_A` overrides `make()` and returns 1, but `Sub_B` inherits the base version, whose body is `pass` - so it returns `None`. `1 + None` raises `TypeError`.

83.

Assuming that the code below has been executed successfully, which of the expressions evaluate to True?

```
class Class:
    var=1
    def __init__(self, value):
        self.prop=value
Object=Class(2)
```

A. 'var' in Class.__dict__
B. 'var' in Object.__dict__
C. len(Object.__dict__) == 1
D. 'prop' in Class.__dict__

Answer: A, C

`var` is defined in the class body so it lives on the class; `prop` is assigned to `self` so it lives on the instance, which therefore has exactly one entry in its `__dict__`.

84.

What is true about Python class constructors?

A. there can be only one constructor in a Python class
B. the constructor cannot be invoked directly under any circumstances
C. the constructor cannot return a result other than None
D. the constructor's first parameter must always be named self

Answer: B, D

`__init__` is invoked for you by instantiation, never called directly, and its first parameter is conventionally `self`. A second `def __init__` replaces the first rather than overloading it.

85.

Assuming that the following inheritance set is in force, which of the following classes are declared properly?

```
class A: pass
class B(A): pass
class C(A): pass
class D(B,C): pass
```

A. class Class_3(A,C): pass
B. class Class_4(C,B): pass
C. class Class_1(D): pass
D. class Class_2(A,B): pass

Answer: A, C

`class Class_3(A, C)` and `class Class_1(D)` both give Python a consistent method resolution order. The failing combinations put a base class before its own subclass, which cannot be linearised.

86.

What is the expected behaviour of this code?

```
class Class:
    Variable=0
    def __init__(self):
        self.value=0
object_1=Class()
Class.Variable+=1
object_2=Class()
object_2.value+=1
print(object_2.Variable+object_1.value)
```

A. It outputs 2
B. It raises an exception
C. It outputs 1
D. It outputs 0

Answer: C

*`Class.Variable += 1` changes the attribute **on the class**, so every instance sees 1. Compare the near-identical question that writes `object_1.Variable += 1` instead - that creates a private copy on one instance and leaves the class at 0.*

87.

What is the expected behaviour of this code?

```
class Upper:
    def __init__(self):
        self.property='upper'
class Lower(Upper):
    def __init__(self):
        super().__init__()
Object=Lower()
print(isinstance(Object,Lower), end='')
print(Object.property)
```

A. True lower
B. True upper
C. False upper
D. False lower

Answer: B

`Lower` derives from `Upper` and its `__init__` calls `super().__init__()`, so `property` does get set and reads `'upper'`. The object genuinely is a `Lower`, so `isinstance` is True.

88.

What is the expected behaviour of this code?

```
class ClassA:
    var=1
    def __init__(self,prop):
        prop1=prop2=prop
class ClassB(ClassA):
    def __init__(self,prop):
        prop3=prop**2
        super().__init__(prop)
    def __str__(self):
        return 'Object'
Object=ClassA(2)
```

A. ClassA.__module__ == '__main__'
B. __name__ == '__main__'
C. str(Object) == 'Object'
D. len(ClassB.__bases__) == 2

Answer: A, B

This question's key was wrong and is corrected here. *`__str__` is defined on `ClassB`, but `Object` is a `ClassA`, so `str(Object)` gives the default `<__main__.ClassA object ...>` - option C is false. `ClassB.__bases__` is `(ClassA,)`, length 1, so D is false too. The two true statements are A and B.*

`ClassA.__module__` is `'__main__'` when the file is the one being run. Watch the `__str__`: it is defined on `ClassB`, and `Object` is a `ClassA`, so it does not apply.

89.

Scenario: You are developing a game in Python and want to represent different characters as objects. Which OOP concept will you use?

A. Inheritance
B. Encapsulation
C. Polymorphism
D. Abstraction

Answer: C

Unicode is a standard that assigns a number, called a code point, to every character in every writing system, plus encodings such as UTF-8 that store those numbers as bytes.

Domain 4 — Modules and Libraries (17)

90.

Can a module run like regular code?

A. yes, and it can differentiate its behaviour between the regular launch and import
B. it depends on the Python version
C. yes, but it cannot differentiate its behaviour between the regular launch and import
D. no, it is not possible; a module can be imported, not run

Answer: A

A module is just a Python file, so it can be run directly. The `__name__` variable is `'__main__'` when it is run and the module's own name when it is imported, which is exactly how a file tells the two apart.

91.

A file name like services.cpython-36.pyc says that:

A. the interpreter used to generate the file is version 3.6
B. it has been produced by CPython
C. it is the 36th version of the file

D. the file comes from the services.py source file

Answer: A, B, D

The name encodes three things: the source file (`services.py`), the implementation that compiled it (CPython) and that implementation's version (3.6). It is not a revision number.

92.

What can you do if you don't like a long package path like import alpha.beta.gamma.delta.epsilon.zeta?

A. you can make an alias for the name using the alias keyword
B. nothing, you need to come to terms with it
C. you can shorten it to alpha.zeta and Python will find the proper connection
D. you can make an alias for the name using the as keyword

Answer: D

`import a.b.c as short` binds the shorter name. There is no `alias` keyword, and Python will not guess at a shortened path.

93.

What should you put instead of XXX to print out the module name?

```
if __name__ != 'XXX':
    print(__name__)
```

A. main
B. _main
C. __main__
D. _main_

Answer: C

Python sets `__name__` to the string `'__main__'` in the file being run directly, and to the module's own name when it is imported. Note the two underscores on each side.

94.

Files with the suffix .py contain:

A. Python source code
B. backups
C. temporary data
D. semi-compiled Python code

Answer: A

`.py` is Python source. `.pyc` holds the semi-compiled bytecode, which is what D describes.

95.

Package source directories/folders can be:

A. converted into the so-called pycK format
B. packed as a ZIP file and distributed as one file
C. rebuilt to a flat form and distributed as one directory/folder
D. removed as Python compiles them into an internal portable format

Answer: B

A package is a directory tree, and it can be zipped and shipped as one file - which is what a wheel is underneath.

96.

What can you deduce from the line below?

```
x = a.b.c.f()
```

A. import a.b.c should be placed before that line
B. f() is located in subpackage c of subpackage b of package a

C. the line is incorrect
D. the function being invoked is called a.b.c.f()

Answer: A, B, D

The stem says choose two, but the bank keys three (A, B and D). All three are in fact correct, so the count in the stem is the error.

`x = a.b.c.f()` calls `f` from subpackage `c`, inside `b`, inside package `a`, so the full dotted path is the function's name and the package has to be imported first for the lookup to resolve.

97.

Assuming that the code below has been executed successfully, which of the following expressions will always evaluate to True?

```
import random
random.seed(1)
v1=random.random()
random.seed(1)
v2=random.random()
```

A. v1 == v2
B. v1 > v2
C. len(random.sample([1,2,3],2)) > 2
D. random.choice([1,2,3]) >= 1

Answer: A, D

Seeding with the same value makes the sequence repeat, so `v1 == v2` always holds. `random.choice` on `[1,2,3]` can only return one of those, all >= 1. `sample` cannot return more items than you asked for.

98.

Which one of the platform module functions should be used to determine the underlying platform name?

A. platform.python_version()
B. platform.processor()
C. platform.platform()
D. platform.uname()

Answer: C

`platform.platform()` returns the whole platform string. `uname()` returns a structured tuple, and `processor()` only the CPU.

99.

What is the expected output of the following code?

```
import sys
import math
b1=type(dir(math)[0]) is str
b2=type(sys.path[-1]) is str
print(b1 and b2)
```

A. FALSE
B. None
C. TRUE
D. 0

Answer: C

`dir()` returns a list of names as strings and `sys.path` is a list of directory strings, so both checks are true and and gives True. (The option is spelled TRUE here; Python prints `True`.)

100.

With regards to the directory structure, select the proper forms of the directives in order to import module_a.

A. from pypack import module_a
B. import module_a from pypack

C. import module_a
D. import pypack.module_a

Answer: A, D

Either name the package on the way in - `from pypack import module_a` - or import the full dotted path. A bare `import module_a` only works if the module's own directory is on the path.

101.

A Python module named pymod.py contains a function named python(). Which of the following snippets will let you invoke the function?

A. import pymod; pymod.python()
B. from pymod import python; python()
C. from pymod import *; pymod.python()
D. import python from pymod; python()

Answer: A, B

`import pymod` binds the module, so the function is reached as `pymod.python()`. `from pymod import python` binds the function itself, so it is called bare. Option C mixes the two and would fail.

102.

What is true about Python packages?

A. a package is a single file whose name ends with the .pa extension
B. a package is a group of related modules
C. the __name__ variable always contains the name of a package
D. the .pyc extension is used to mark semi-compiled Python packages

Answer: B, C

*A package is a group of related modules in a directory. `__name__` holds the *module's* name - and `'__main__'` in the file being run - so it is not always a package name; `.pyc` marks compiled modules, not packages.*

103.

A Python module named pymod.py contains a variable named pymod. Which of the following snippets will let you access the variable?

A. import pymod; pymod.pyvar = 1
B. import pymod from pymod; pymod = 1
C. from pymod import pymod; pymod()
D. from pymod import pymod; pymod = 1

Answer: C, D

`from pymod import pymod` binds the module's variable into your namespace, which is how you reach it. Note that rebinding that name afterwards affects only your copy, not the module's.

104.

Assuming that the code below has been executed successfully, which of the following expressions will always evaluate to True?

```
import random
v1=random.random()
v2=random.random()
```

A. v1 == v2
B. v1 < 1
C. random.choice([1,2,3]) > 0
D. len(random.sample([1,2,3],1)) > 2

Answer: B, C

`random.random()` returns a float in [0.0, 1.0), so it is always below 1, and `choice` on `[1,2,3]` always returns one of those. Two unseeded calls are not expected to be equal, and `sample(seq, 1)` returns exactly one item.

105.

With regards to the directory structure, select the proper forms of the directives in order to import module_b.

A. from pyback.upper import module_b
B. import pyback.upper.module_b
C. import upper.module_b
D. import module_b

Answer: B

The stem says choose two but the bank keys only one.

The full dotted path always works: `import pyback.upper.module_b`. A partial path like `upper.module_b` only resolves if that directory is itself on the import path.

106.

What is the expected behaviour of this code?

```
import sys
b1=type(dir(sys)) is str
b2=type(sys.path[-1]) is str
print(b1 and b2)
```

A. FALSE
B. 0
C. None
D. TRUE

Answer: A

This question's key is malformed and is corrected here. *It reads `FALSE` - the option *text* rather than a letter, almost certainly Excel coercing "False" into a boolean. It maps to option A.*

*`dir(sys)` returns a **list**, not a str, so `b1` is False and `b1 and b2` is False without even evaluating `b2`. That short-circuit is why `and` stops at the first falsy operand.*

Domain 5 — Debugging and Error Handling (16)

107.

What is the expected output of the following snippet?

```
s='abc'
for i in len(s):
    s[i]=s[i].upper()
print(s)
```

A. abc
B. The code will cause a runtime exception
C. ABC
D. 123

Answer: B

`len(s)` is an integer and integers are not iterable, so `for i in len(s)` raises `TypeError` before anything else happens. You want `for i in range(len(s))` - and even then, strings are immutable so `s[i] = ...` would fail too.

108.

What is the expected behaviour of the following snippet?

```
def a(l,l):
    return l[l]
print(a(0,[1]))
```

A. cause a runtime exception
B. print 1

C. print 0, [1]
D. SyntaxError

Answer: D

*Two parameters with the same name is a `SyntaxError`, caught when the file is compiled - so this fails before any line runs. That is why D is right and A (a *runtime* exception) is not.*

109.

How to handle an exception and assign it to variable e?

A. except Exception (e):
B. except e = Exception:
C. except Exception as e:
D. such an action is not possible in Python

Answer: C

`except SomeError as e:` binds the exception instance to e. The comma form was Python 2 and is now a syntax error.

110.

What is the expected output of the following code?

```
lst=[x for x in range(5)]
lst=list(filter(lambda x: x%2==0, lst))
print(len(lst))
```

A. 2
B. The code will cause a runtime exception
C. 1
D. SyntaxError

Answer: A

This question is broken. *`[x for x in range(5)]` is `[0,1,2,3,4]`; filtering for `x % 2 == 0` keeps 0, 2 and 4, so `len` is **3**. No option offers 3.*

`range(5)` is 0 to 4. Keeping the even ones leaves 0, 2 and 4, so the length is 3. Remember that 0 is even.

111.

What is the expected behaviour of the following code?

```
def unclear(x):
    if x%2==1:
        return 0
print(unclear(1)+unclear(2))
```

A. print 0
B. cause a runtime exception
C. prints 3
D. print an empty line

Answer: B

The function returns 0 only when the argument is odd; with an even argument it falls off the end and returns `None`. `0 + None` raises `TypeError`. A function with a conditional `return` returns `None` on every path you did not cover.

112.

If you need to serve two different exceptions called Ex1 and Ex2 in one except branch, you can write:

A. except Ex1 Ex2:
B. except (Ex1, Ex2):
C. except Ex1, Ex2:
D. except Ex1+Ex2:

Answer: B

Multiple exception types go in a parenthesised tuple. The bare comma form was Python 2 syntax for binding the instance and is now a syntax error.

113.

What is the expected behaviour of the following code? (try/except missing colon)

A. it outputs 3
B. it outputs 1
C. the code is erroneous and it will not execute
D. it outputs 2

Answer: C

A missing colon is a `SyntaxError`, raised when the file is compiled, so nothing runs at all and no output appears.

114.

What is the expected behaviour of the following code?

```
string=str(1/3)
dummy=''
for character in strong:
    dummy=character+dummy
print(dummy[-1])
```

A. it raises an exception
B. it outputs 0
C. it outputs 3
D. it outputs None

Answer: A

`strong` is a typo for `string`, and the name was never assigned, so the loop raises `NameError` before anything prints. Python only catches this when the line actually runs.

115.

What is the expected behaviour of this code?

```
my_list=[i for i in range(5)]
m=[my_list[i] for i in range(4,0,-1)] if my_list[i]%2!=0]
print(m)
```

A. it outputs [1,3]
B. the code is erroneous and it will not execute
C. it outputs [3,1]
D. it outputs [4,2,0]

Answer: B

The comprehension has an unbalanced bracket, so it is a `SyntaxError` and nothing runs. Read the brackets rather than the logic when a snippet looks odd.

116.

Which of the following snippets will execute without raising any unhandled exceptions?

A. try: print(float('1e1')) except (ValueError,NameError): print(float('1a1')) else: print(float('101'))
B. try: print(1/1) except: print(2/1) else: print(3/0)
C. try: print(1/0) except ValueError: print(1/1) else: print(1/2)
D. try: print(0/1) except: print(1/1) else: print(2/1)

Answer: A, D

A works because `float('1e1')` is valid, so no exception occurs and the `else` runs `float('101')`, also valid. D works because `0/1` is fine. B ends with `3/0` in its `else`, and C's `except ValueError` cannot catch the `ZeroDivisionError` that `1/0` raises.

117.

What is the expected behaviour of this code?

```
my_list=[i for i in range(5,0,-1)]
m=[my_list[i] for i in range(5)] if my_list[i]%2==0]
print(m)
```

A. it outputs [4,2]
B. it outputs [2,4]
C. it outputs [0,1,2,3,4]
D. the code is erroneous and it will not execute

Answer: D

The comprehension has an unmatched `]`, so this is a `SyntaxError` and nothing runs at all. When a snippet looks strange, count the brackets before reasoning about the logic.

118.

What is the expected behaviour of this code?

```
the_list='1,2 3'.split()
the_string=''.join(the_list)
print(the_string.insight())
```

A. it raises an exception
B. it outputs nothing
C. it outputs True
D. it outputs False

Answer: A

`.split()` and `.join()` both work fine, but `.insight()` is not a string method - it does not exist - so the code raises `AttributeError` before `print` gets anything. Nothing is output because the program stops there.

119.

What is the expected behaviour of this code?

```
s='2A'
try:
    n=int(s)
except ValueError:
    n=2
except ArithmeticError:
    n=1
except:
    n=0
print(n)
```

A. it outputs 0
B. the code is erroneous and it will not execute
C. it outputs 1
D. it outputs 2

Answer: D

`int('2A')` raises `ValueError`, and handlers are tried in order, so the first matching one wins and n becomes 2. The later branches never run.

120.

Which of the following snippets will execute without raising any unhandled exceptions?

A. try: print(0/0) except: print(0/1) else: print(0/2)
B. try: print(int('0')) except NameError: print('0') else: print(int(''))
C. import math try: print(math.sqrt(-1)) except: print(math.sqrt(0)) else: print(math.sqrt(1))
D. try: print(float('1e1')) except (NameError, SystemError): print(float('1a1')) else: print(float('1c1'))

Answer: A, C

*A and C both raise inside the `try` and catch it with a bare `except`, whose body succeeds - so nothing escapes. B and D succeed in the `try`, which means their `else` runs, and both `else` bodies raise (`int('')` and `float('1c1')`) with no handler left. An `else` block is **not** protected by the `except` above it.*

121.

What is the expected behaviour of this code?

```
my_list=[1,2,3]
try:
    my_list[3]=my_list[2]
except BaseException as error:
    print(error)
```

A. it outputs error
B. it outputs <class 'IndexError'>
C. it outputs list assignment index out of range
D. the code is erroneous and it will not execute

Answer: C

*`print(error)` prints the exception's *message*, not its class. `my_list[3]` is past the end of a three-element list, so the message is `list assignment index out of range`. Printing `type(error)` is what would give you the class.*

122.

What is the expected behaviour of this code?

```
class E(Exception):
    def __init__(self,message):
        self.message=message
    def __str__(self):
        return 'it''s nice to see you'
try:
    print('I feel fine')
    raise Exception('what a pity')
except E as e:
    print(e)
else:
    print('the show must go on')
```

A. the string 'it's nice to see you' will be seen
B. the string 'the show must go on' will be printed
C. the string 'I feel fine' will be printed
D. nothing will be printed

Answer: C

`raise Exception(...)` raises the base class, but the handler only catches E, so it does not match and the exception is unhandled. `'I feel fine'` has already printed by then; the `else` never runs because the `try` did not complete.

Domain 6 — File Handling (11)

123.

There is a stream named s open for writing. What option will you select to write a line to the stream?

A. s.write('Hello\n')
B. write(s,'Hello')
C. s.writeln('Hello')
D. s.writeln('Hello')

Answer: A

Options C and D are identical, so one is a typo. It does not affect the answer: there is no `writeln` in Python at all.

File objects have `write()`, which adds nothing of its own - you supply the newline. There is no `writeln` in Python.

124.

You are going to read just one character from a stream called s. Which statement would you use?

A. ch = read(s,1)
B. ch = s.input(1)
C. ch = input(s,1)
D. ch = s.read(1)

Answer: D

`read(n)` reads at most n characters, so `s.read(1)` gets exactly one. `input()` reads from the keyboard, not from a file object.

125.

What can you deduce from the following statement?

```
str = open('file.txt','rt')
```

A. str is a string read in from the file named file.txt
B. a newline character translation will be performed during the reads
C. if file.txt does not exist, it will be created
D. the opened file cannot be written with the use of the str variable

Answer: B, D

`'rt'` is read + text mode. Text mode translates newlines on the way in, and read mode means the object cannot be written through. Opening for reading never creates a missing file - it raises `FileNotFoundError`.

126.

What does the open() function return?

A. an integer value identifying an opened file
B. an error code (0 means success)
C. a stream object
D. always None

Answer: C

`open()` returns a file object, also called a stream. Errors are reported by raising, not by a return code.

127.

If S is a stream open for reading, what do you expect from the following invocation?

```
c = s.read()
```

A. one line of the file will be read and stored in the string called c
B. the whole file content will be read and stored in the string called c
C. one character will be read and stored in the string called c
D. one disk sector (512 bytes) will be read and stored in the string called c

Answer: B

*`read()` with no argument consumes the **whole** remaining file into one string. `read(1)` gets one character and `readline()` gets one line.*

128.

You are going to read 16 bytes from a binary file into a bytearray called data. Which lines would you use?

A. data=bytearray(16); bf.readinto(data)
B. data=binfile.read(bytearray(16))
C. bf.readinto(data=bytearray(16))
D. data=bytearray(binfile.read(16))

Answer: A, D

`readinto()` fills a buffer you already made; `read(n)` returns fresh bytes you can wrap in a `bytearray`. Both give you 16 bytes - the difference is who allocates.

129.

What is the expected behaviour of this code?

```
try:
    f=open('non_existing_file','w')
    print(1,end=' ')
    s=f.readline()
    print(2,end=' ')
except IOError as error:
    print(3,end=' ')
else:
    f.close()
    print(4,end=' ')
```

A. 124
B. 1234
C. 24
D. 13

Answer: D

This question's key was wrong and is corrected here. *The bank keys `124`; running it prints `1 3`.*

*The bank keys **`124`**; it actually prints **`1 3`**. Opening in `'w'` mode succeeds and creates the file, so `print(1)` runs. Then `readline()` on a write-only handle raises `io.UnsupportedOperation`, which subclasses `OSError` and so IS caught by `except IOError`, printing 3. The `else` never runs, because `else` means the `try` finished without an exception.*

130.

What is the expected behaviour of this code?

```
try:
    f=open('existing_text_file','w')
    d=f.readlines()
    print(len(d))
    f.close()
except IOError:
    print(-1)
```

A. the length of the first line from the file
B. -1
C. the number of lines contained inside the file
D. the length of the last line from the file

Answer: B

This question's key is unusable and is corrected here. *It reads `0 (none of the above)`, which is not one of the four options.*

*It prints **`-1`**. `'w'` mode truncates the file, then `readlines()` on a write-only handle raises `io.UnsupportedOperation` - a subclass of `OSError` - so the handler catches it.*

131.

Which of the following statements are true?

A. if invoking open() fails, an exception is raised
B. open() requires a second argument
C. open() is a function which returns an object that represents a physical file
D. instd, outstd, errstd are the names of pre-opened streams

Answer: A, C

`open()` raises on failure rather than returning an error code, and what it returns represents the file. The mode argument is optional and defaults to `'r'`; the pre-opened streams are `stdin`, `stdout` and `stderr`.

132.

What is the expected behaviour of this code?

```
try:
    f=open('non_zero_length_existing_text_file','rt')
    d=f.read(1)
    print(len(d))
    f.close()
except IOError:
    print(-1)
```

A. 1
B. 0
C. an error value corresponding to file not found
D. 1

Answer: A

Options A and D are identical (1), so one is a typo. The answer is 1: `read(1)` on a non-empty existing file returns one character, and `len` of that is 1.

`read(1)` returns one character from a non-empty file, so `len(d)` is 1. The `except` branch would only run if the file were missing.

133.

Story: Sarah is writing a Python program that reads data from a file. Which of the following modes should she use to open the file for reading?

A. 'w'
B. 'r'
C. 'a'
D. 'x'

Answer: B

A missing colon after a `class` header is a `SyntaxError`, so the file never runs.